

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Milestone 3 Documentation

Darapuneni Bhavya

TABLE OF CONTENTS

S.No	Title	Page No.
1	Introduction	1
2	Objectives of Milestone 3	2
3	YOLO Dataset Preparation	3
4	YOLO Object Detection Model Training	4-5
5	YOLO Model Integration	6-8
6	Object Detection Workflow	9-10
7	Severity Scoring	11
8	Quality Risk Assessment	12-13
9	Production Quality Report	14-15
10	Conclusion	16

1. Introduction

The **VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System** was initially developed using an image classification approach to identify whether a manufacturing product was **Defective** or **No Defect**. Although the classification model provided prediction results, it was not capable of identifying the exact location of defects within the image.

To improve the inspection process, **Milestone 3** extends the system by integrating the **YOLOv8 (You Only Look Once Version 8)** object detection model. Unlike image classification, object detection identifies the regions in an image where defects are present by generating bounding boxes around the detected areas. This provides better visual understanding of the inspection results and supports more effective quality analysis.

In this milestone, the trained YOLOv8 model is integrated with the FastAPI backend to perform real-time object detection on uploaded manufacturing images. The model processes the input image and returns the detected defect regions along with confidence scores. Based on the confidence score, the system determines the severity level and generates quality assessment information for the inspected product.

Milestone 3 also introduces additional inspection features such as severity scoring, quality assessment, production quality reporting, dashboard analytics, and trend monitoring. These features help users understand inspection results more effectively while improving the overall manufacturing defect inspection process.

Overall, Milestone 3 enhances the VisionInspect AI system by combining object detection with quality assessment and reporting, making the inspection process more informative, accurate, and suitable for real-time manufacturing environments.

2. Objectives of Milestone 3

The primary objective of Milestone 3 is to enhance the VisionInspect AI system by integrating an object detection model and implementing advanced quality inspection and reporting features. The specific objectives are:

- To integrate the YOLOv8 object detection model for manufacturing image inspection.
- To detect defect regions in uploaded manufacturing images.
- To display detected defect regions using bounding boxes and confidence scores.
- To perform severity scoring based on the confidence score generated by the object detection model.
- To implement quality assessment for inspected manufacturing products.
- To generate production quality reports containing inspection details and prediction results.
- To develop dashboard analytics for monitoring inspection statistics.
- To implement trend monitoring workflows for analyzing inspection history.
- To improve the efficiency and reliability of manufacturing defect inspection through AI-based object detection.

3. YOLO Dataset Preparation

Dataset preparation is an important step in developing an object detection model. In the VisionInspect AI project, the **prepare_yolo_dataset.py** script is implemented to organize the manufacturing dataset into a format required by the YOLOv8 object detection framework.

The script automatically reads the prepared dataset, creates the required folder structure for object detection, and copies the corresponding images and annotation files into their respective directories. It also generates the **dataset.yaml** configuration file, which contains the dataset path, training and validation directories, number of classes, and class names required for YOLO model training.

This organized dataset structure enables the YOLOv8 model to efficiently access the training and validation data during the training process.

3.1 YOLO Dataset Preparation Workflow

1. Load the prepared manufacturing dataset.
2. Create the required YOLO dataset folder structure.
3. Copy the training images and annotation files.
4. Copy the validation images and annotation files.
5. Generate the **dataset.yaml** configuration file.
6. Save the prepared YOLO dataset for model training.

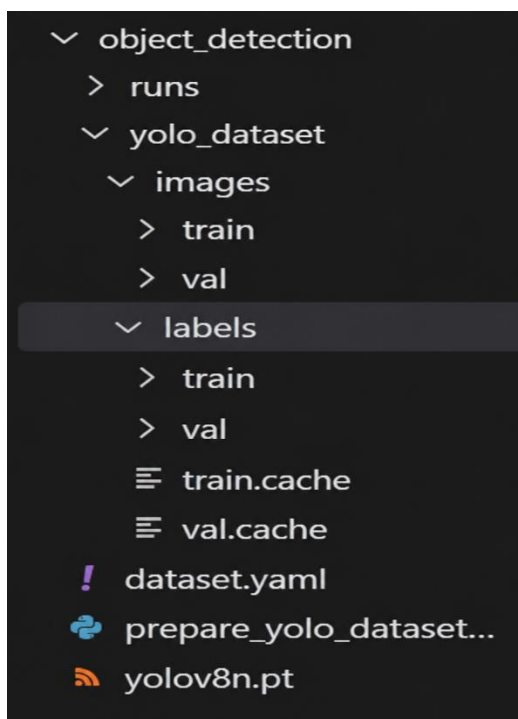


Figure 3.1: YOLO Dataset Folder Structure

Figure 3.1 illustrates the folder structure generated by the `prepare_yolo_dataset.py` script. The dataset is organized into separate **images** and **labels** directories, each containing **train** and **validation** folders. The `dataset.yaml` file stores the dataset configuration required for YOLOv8 training. This structured organization ensures that the object detection model can efficiently access the training and validation data during the training process.

4. YOLO Object Detection Model Training

4.1 YOLO Model Training

After preparing the dataset in the required YOLO format, the **YOLOv8** object detection model was trained using the **Ultralytics** framework. The prepared dataset and the dataset.yaml configuration file were used to load the training and validation images during the training process.

The model was trained over multiple epochs to learn the visual characteristics of defect regions present in manufacturing images. During each training epoch, the model continuously updated its weights to improve its object detection performance. The training process automatically monitored the model's learning progress using various performance metrics, including training loss, validation loss, precision, recall, and Mean Average Precision (mAP).

At the end of training, the best-performing model weights were automatically saved as **best.pt**. This trained model was later integrated with the FastAPI backend to perform real-time object detection on uploaded manufacturing images. The training process also generated performance graphs that were used to evaluate the learning behaviour and overall detection capability of the YOLOv8 model.

4.2 YOLO Training Results

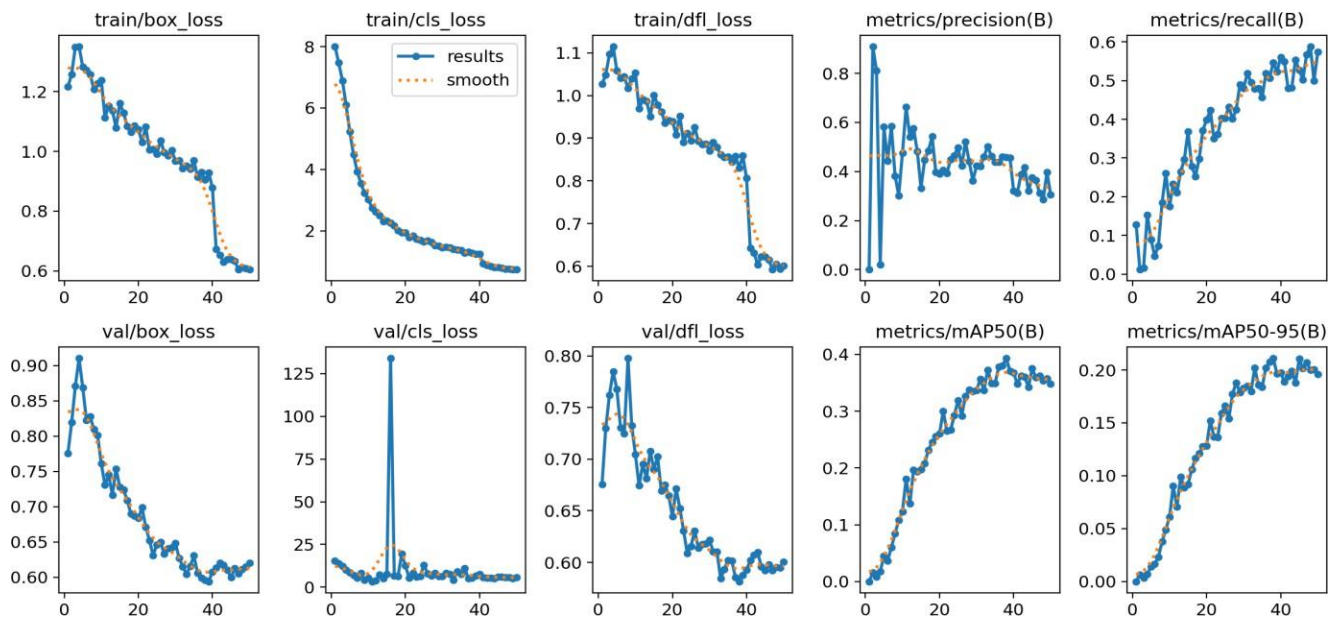


Figure illustrates the performance graphs generated during the YOLOv8 training process. The graphs show the variation of training loss, validation loss, precision, recall, and Mean Average Precision (mAP) over multiple training epochs. The decreasing loss values indicate that the model gradually learned the defect patterns from the training dataset, while the improvement in recall and mAP demonstrates better object detection performance. These metrics were used to monitor the learning progress and evaluate the overall effectiveness of the trained YOLOv8 model.

4.3 Model Performance

The performance of the trained YOLOv8 object detection model was evaluated using the validation dataset. The evaluation metrics generated during training indicate the model's ability to detect defect regions in manufacturing images. The best-performing model was automatically saved as **best.pt**, which was later integrated into the application for real-time manufacturing object detection.

Performance Metric	Value
Model	YOLOv8
Saved Model	best.pt
Training Framework	Ultralytics
Dataset Configuration	dataset.yaml
Detection Method	Object Detection

5. YOLO Model Integration

The trained YOLOv8 object detection model is integrated with the FastAPI backend to perform real-time manufacturing object detection. After the training process is completed, the best-performing model is saved as **best.pt** and loaded into the backend using the Ultralytics YOLO library.

Whenever a manufacturing image is uploaded through the application, the backend first performs image preprocessing and then passes the processed image to the trained YOLOv8 model for prediction. The model analyzes the uploaded image, detects objects, generates bounding boxes around the detected regions, and calculates the confidence score for each detected object.

The prediction results generated by the YOLOv8 model are processed by the FastAPI backend and sent to the frontend application through the API. The frontend displays the processed image together with the prediction result, detected category, confidence score, severity level, risk level, and object detection results, enabling users to perform real-time manufacturing inspection.

The confidence scores obtained from the YOLOv8 model are further used for **severity scoring, quality risk assessment, and production quality report generation**, providing users with a complete quality inspection workflow.

5.1 YOLO Model Integration Workflow

1. Load the trained **best.pt** model.
2. Receive the uploaded manufacturing image.
3. Perform image preprocessing.
4. Pass the processed image to the YOLOv8 model.
5. Detect objects in the uploaded image.
6. Generate bounding boxes and confidence scores.
7. Return the detection results to the FastAPI backend.
8. Send the processed inspection results to the frontend application.
9. Display the processed image, detected category, confidence score, severity level, risk level, and object detection results to the user.


```

from ultralytics import YOLO
import os
import cv2

MODEL_PATH = os.path.abspath(
    os.path.join(
        os.path.dirname(__file__),
        "../../object_detection/runs/detect/train/weights/best.pt"
    )
)

model = YOLO(MODEL_PATH)

def predict_objects(image_path):

    results = model.predict(
        source=image_path,
        conf=0.15,
        save=False
    )

    image = cv2.imread(image_path)

    detections = []

    for result in results:

        for box in result.boxes:

            cls = int(box.cls[0])
            conf = float(box.conf[0])

            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())

            label = model.names[cls]

            detections.append({
                "class": label,
                "confidence": round(conf * 100, 2),
                "bbox": [x1, y1, x2, y2]
            })

```

Figure 5.1: Implementation of yolo_model.py

Figure 5.1 illustrates the implementation of the yolo_model.py module used for integrating the trained YOLOv8 object detection model into the VisionInspect AI system. The module loads the trained **best.pt** model, receives the uploaded manufacturing image, performs object detection using the predict() function, extracts the detected object class, confidence score, and bounding box coordinates, and returns the detection results to the FastAPI backend. These results are further used for frontend visualization, severity scoring, quality risk assessment, and production quality report generation.

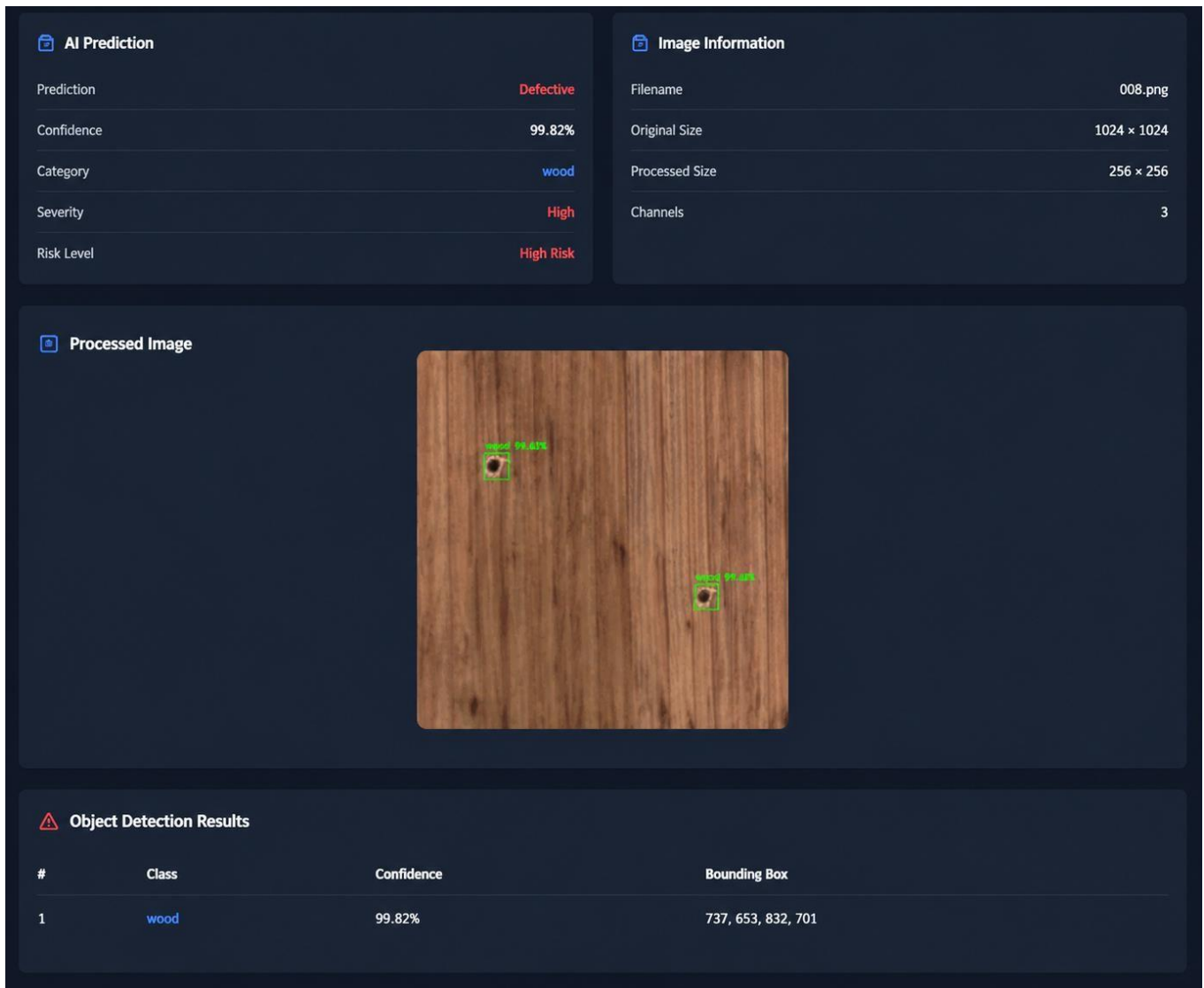


Figure 5.2: YOLO Model Output in the Application

Figure 5.2 illustrates the output generated after integrating the trained YOLOv8 object detection model with the VisionInspect AI application. After the uploaded manufacturing image is processed, the frontend displays the AI prediction, image information, processed image with detected object bounding boxes, and object detection results. It also presents the detected category, confidence score, severity level, and risk level, demonstrating the successful integration of the YOLOv8 model with both the FastAPI backend and the frontend application.

6. Object Detection Workflow

Object detection is one of the major functionalities introduced in Milestone 3. After integrating the trained YOLOv8 model with the FastAPI backend, the application performs real-time object detection on uploaded manufacturing images.

Whenever a user uploads a manufacturing image, the backend first performs image preprocessing and then passes the processed image to the trained YOLOv8 model. The model analyzes the image and detects defect regions by generating bounding boxes around the identified areas. Along with each detected region, the model also calculates a confidence score that indicates the reliability of the prediction.

The detected object information, including the confidence score and bounding box coordinates, is returned to the frontend application and displayed to the user. This enables users to visually inspect the detected regions and better understand the inspection results.

6.1 Object Detection Workflow

1. Upload the manufacturing image.
2. Perform image preprocessing.
3. Load the trained YOLOv8 model.
4. Detect defect regions in the uploaded image.
5. Generate bounding boxes around detected regions.
6. Calculate the confidence score.
7. Return the detection results to the backend.
8. Display the detected output in the frontend application.

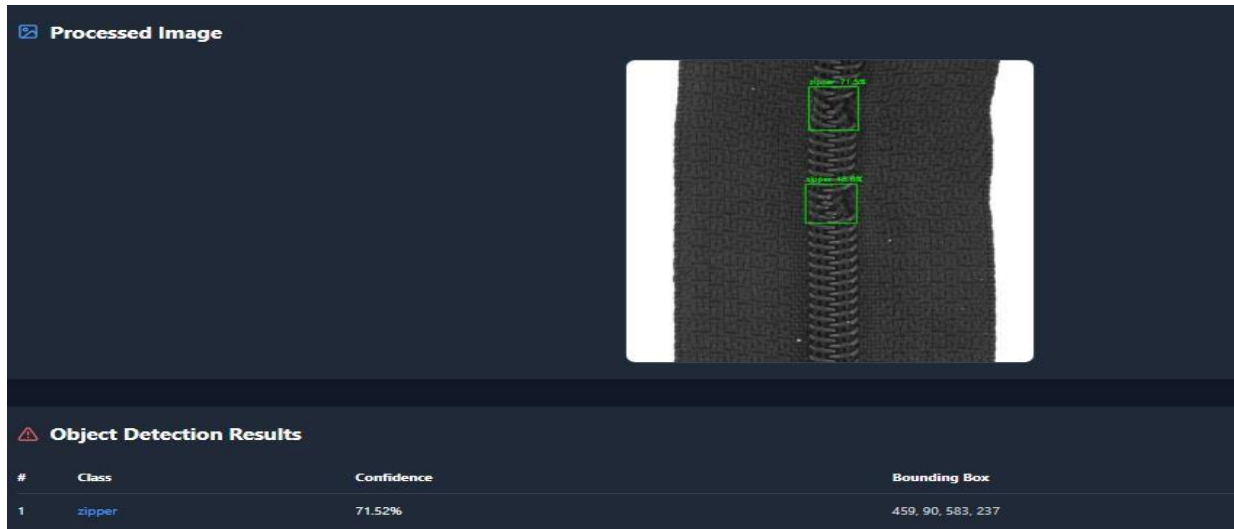


Figure 6.1: Object Detection Output

Figure 6.1 illustrates the object detection output generated by the VisionInspect AI application. The uploaded manufacturing image is processed by the YOLOv8 model, and the detected region is highlighted using a bounding box. The application also displays the corresponding confidence score and the bounding box coordinates in the Object Detection Results table, providing users with a clear visualization of the detection outcome.

7. Severity Scoring

Severity scoring is implemented to indicate the significance of the detected object based on the confidence score generated by the YOLOv8 object detection model. After object detection is completed, the application selects the detection with the highest confidence score and uses it to determine the severity level.

The confidence score represents the reliability of the detected object. Based on predefined confidence score thresholds, the application automatically categorizes the detection into **Low**, **Medium**, or **High** severity levels. This enables users to quickly understand the confidence of the detection result and supports further quality assessment.

The calculated severity level is displayed along with the object detection results and is also used during quality risk assessment and production quality reporting.

7.1 Severity Scoring Criteria

The application assigns the severity level according to the confidence score returned by the YOLOv8 object detection model.

Confidence Score	Severity Level
Less than 50%	Low
50% – 79.99%	Medium
80% and above	High

The severity level provides users with a simple representation of the confidence of the detected object and assists in evaluating the inspection results.

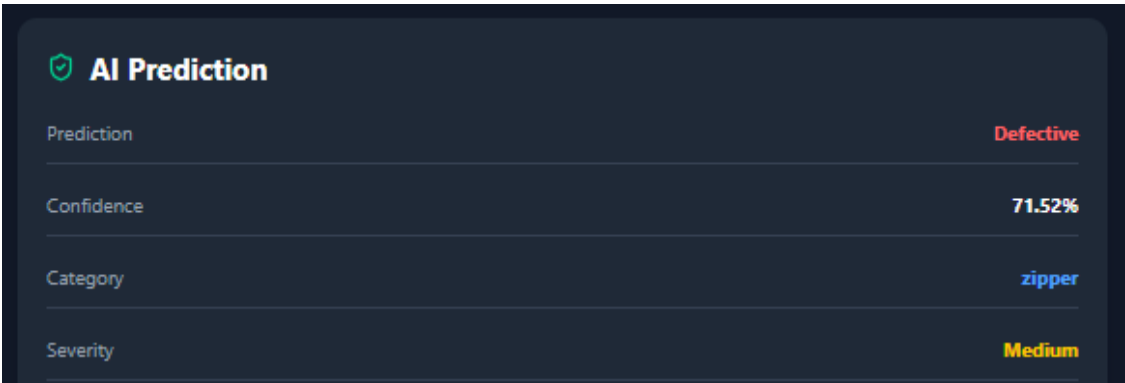


Figure 7.1: Severity Scoring Result

Figure 7.1 illustrates the severity scoring result generated by the VisionInspect AI application. After the YOLOv8 object detection model analyzes the uploaded manufacturing image, the application identifies the detected object and calculates its confidence score. Based on the predefined confidence score thresholds, the system automatically assigns a severity level (**Low, Medium, or High**). These results are displayed in the AI Prediction panel to help users quickly evaluate the significance of the detected object and support further quality assessment.

8. Quality Risk Assessment

Quality Risk Assessment is implemented to help users understand the significance of the detected object based on the confidence score generated by the YOLOv8 object detection model. After object detection is completed, the application automatically evaluates the confidence score and assigns an appropriate risk level.

The application categorizes the inspection result into **Low Risk**, **Medium Risk**, or **High Risk** using predefined confidence score thresholds. Based on the assigned risk level, the system also displays an assessment message and provides appropriate recommendations to assist users during the quality inspection process.

For **High Risk**, the application recommends immediate quality inspection and reviewing the manufacturing process. For **Medium Risk**, it recommends careful inspection, maintenance if necessary, and continuous production monitoring. For **Low Risk**, the application indicates that the product quality is acceptable and recommends continuing regular quality inspection procedures.

The calculated risk level, assessment message, and recommendations are displayed in the **Quality Risk Assessment** section of the application. The same information is also stored in the inspection history and included in the generated production quality report.

8.1 Quality Risk Assessment Criteria

Confidence Score	Risk Level
Less than 50%	Low Risk
50% – 79.99%	Medium Risk
80% and above	High Risk

The application automatically assigns the risk level based on the confidence score returned by the YOLOv8 object detection model, ensuring consistent quality assessment for every inspection.

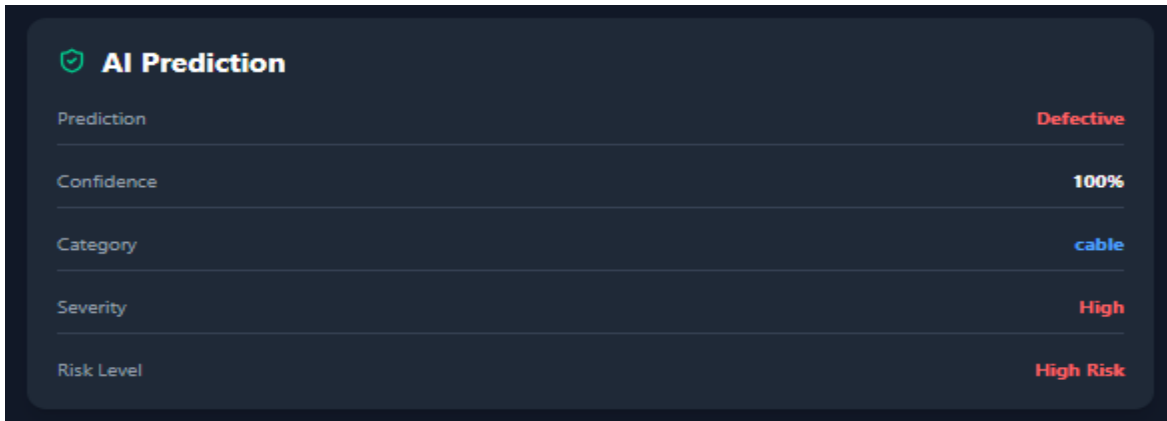


Figure 8.1: AI Prediction Result

Figure 8.1 illustrates the AI Prediction generated by the application after object detection. The prediction result includes the detected object status, confidence score, detected category, severity level, and the corresponding quality risk level. These results provide users with a clear summary of the inspection outcome.

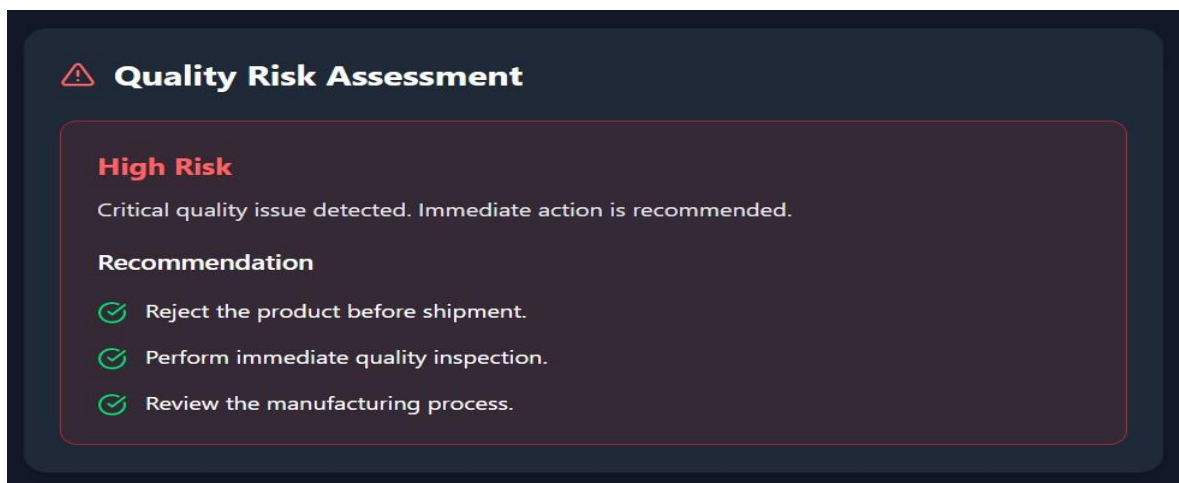


Figure 8.2: Quality Risk Assessment

Figure 8.2 illustrates the Quality Risk Assessment generated by the application. Based on the confidence score returned by the YOLOv8 object detection model, the system automatically assigns the appropriate risk level and displays a corresponding assessment message along with recommendations. These recommendations assist users in determining the appropriate quality control actions and help improve the overall manufacturing inspection process.

9. Production Quality Report

The VisionInspect AI application generates a comprehensive **Production Quality Report** in PDF format after the inspection process is completed. The report summarizes the complete inspection results and provides a structured record of the manufacturing quality assessment.

The report includes the original image, processed image, prediction result, detected category, confidence score, severity level, risk level, object detection results, preprocessing steps, and inspection summary. This information enables users to maintain inspection records, support quality documentation, and review inspection results whenever required.

The report is automatically generated using the **jsPDF** and **jspdf-autotable** libraries. Users can download the inspection report directly from the application by clicking the **Download Inspection Report** button.

9.1 Report Generation Workflow

The application generates the inspection report through the following steps:

1. Create a new PDF document.
2. Add the VisionInspect AI report title and generation date.
3. Insert the original and processed images.
4. Display inspection details such as prediction, category, severity, risk level, and confidence score.
5. Include object detection results and preprocessing steps.
6. Generate the inspection summary.
7. Save the report as **Inspection_Report.pdf**.

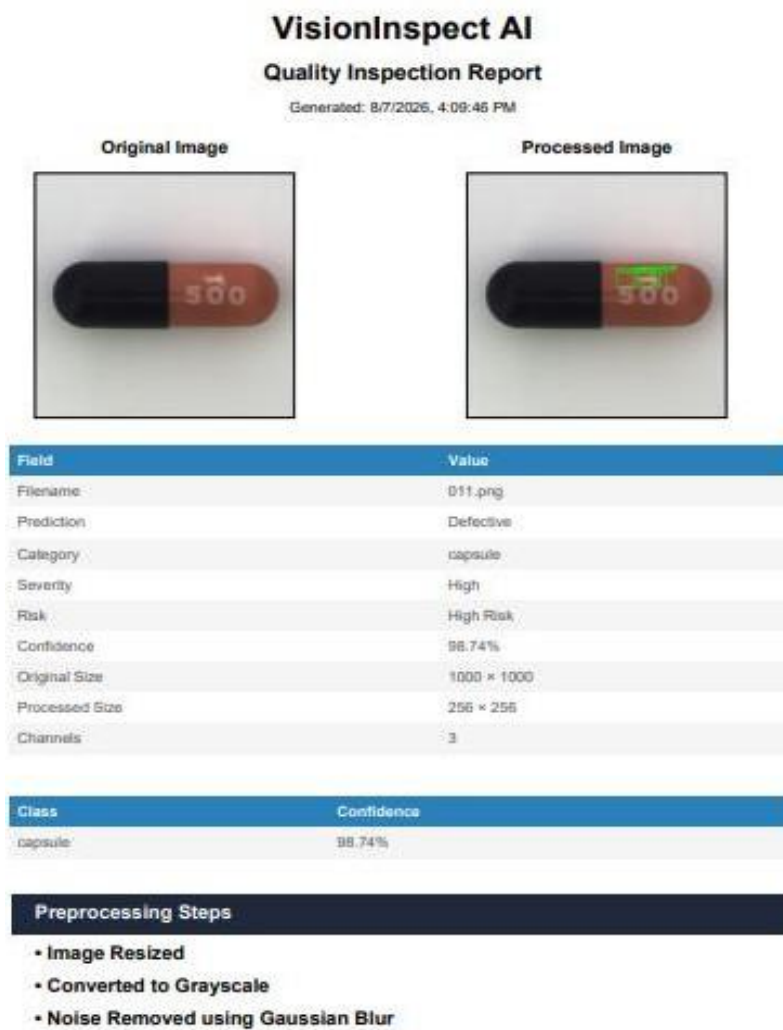


Figure 9.1: Production Quality Report

Figure 9.1 illustrates the Production Quality Report generated by the VisionInspect AI application. The report provides a comprehensive summary of the inspection process by including the original and processed images, prediction result, detected category, confidence score, severity level, risk level, object detection results, and preprocessing details. The generated PDF serves as a complete inspection record and can be downloaded and used for quality documentation and future reference.

10. Conclusion

The VisionInspect AI system was successfully enhanced by integrating the YOLOv8 object detection model into the existing inspection workflow. The developed system is capable of receiving manufacturing images, performing image preprocessing, detecting objects using the trained YOLOv8 model, and displaying the detected objects with bounding boxes and confidence scores. This enables users to identify detected objects quickly and efficiently through an interactive web application.

In addition to object detection, the application automatically evaluates the confidence score returned by the YOLOv8 model and assigns appropriate severity levels and quality risk levels using predefined threshold values. These assessments provide users with a better understanding of the inspection results and support decision-making during the quality inspection process. The generated inspection results are presented through a user-friendly interface, allowing users to easily analyze the prediction, detected category, confidence score, processed image, severity level, and quality risk level.

Furthermore, the application generates a comprehensive production quality report containing the inspection details, processed image, detection results, preprocessing steps, severity level, and quality risk assessment. This report can be downloaded in PDF format and can be used for documentation, quality control, and future reference.

Overall, the integration of YOLOv8 object detection with the FastAPI backend and React frontend has significantly improved the functionality of the VisionInspect AI system. The implemented solution provides a complete AI-powered quality inspection workflow by combining object detection, severity scoring, quality risk assessment, and production quality reporting within a single application. This enhancement contributes to improving manufacturing defect inspection by providing faster, more consistent, and informative inspection results.